

Crypto Order Book Engine

System Design & Architecture Documentation

A production-grade, real-time order book built in Rust. WebSocket ingestion from Binance, async Tokio throughout, REST API with Axum. Every decision documented. Every tradeoff named.

Language: Rust | Runtime: Tokio | Exchange: Binance | API: Axum

Part 1 — What Are We Actually Building

Binance is a crypto exchange. When people buy and sell Bitcoin, those orders sit in a list called an **order book** — bids on one side (people wanting to buy), asks on the other (people wanting to sell). That list changes thousands of times per second.

We are building a program that does three things:

1. Connects to Binance and receives those changes in real time.
2. Keeps its own copy of the order book in memory, always up to date.
3. Lets other programs ask what the order book looks like right now, over HTTP.

Simple goal. Hard to do correctly at production quality. This document explains every decision we made and exactly why we made it.

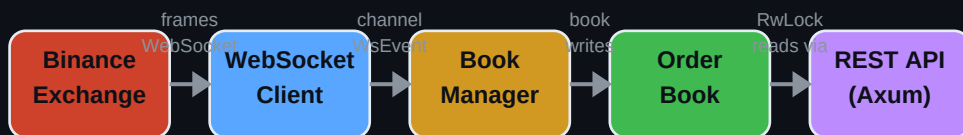


Figure 1 — Top-level data flow

Part 2 — Cargo.toml: The Dependency Decisions

Every dependency is a bet — you're betting this tool is mature, maintained, and won't fight with the rest of your system. Here is every bet we made and why.

tokio

Rust by default does one thing at a time. If you're waiting for Binance to send you a message, your program just sits there doing nothing. Tokio is the system that lets your program do many things at the

same time — listen to Binance, handle an incoming HTTP request, run a timer — without actually needing multiple CPU cores. It is the foundation everything else sits on. Without it, none of the other choices make sense.

tokio-tungstenite

Binance sends us data over a WebSocket. A WebSocket is like a phone call — you open the connection once and both sides can talk back and forth as long as they want, instead of making a new request every time like normal web traffic. tokio-tungstenite speaks that protocol. We picked this one specifically because it is built to work with Tokio — they use the same underlying machinery so they do not fight each other.

futures-util

When Binance sends us a message over the WebSocket connection, it does not arrive all at once with a neat label on it. It arrives as a stream — like water from a tap, not a package being delivered. futures-util gives us the tools to work with that stream: read the next item, process it, wait for the next one.

serde and serde_json

Binance sends us JSON. It looks like this:

```
{"e":"depthUpdate","E":1699000000000,"s":"BTCUSD","U":123456,"u":123460,...}
```

We need to turn that text into actual Rust values we can work with. serde is the system that does that conversion automatically. You describe the shape of what you expect, it reads the JSON and fills in the values. This is the Rust standard — every serious Rust project uses it.

rust_decimal

This one is non-negotiable. Binance sends prices as strings like "29431.50000000". The obvious thing to do is convert that to a floating point number. But floating point numbers cannot represent most decimal values exactly. $0.1 + 0.2$ in floating point does not equal 0.3 — it equals 0.30000000000000004 . In a calculator app that is annoying. In a financial system that is a lawsuit. rust_decimal stores the number exactly as written. We never use floats for prices, ever.

thiserror

When something goes wrong, different things going wrong need different responses. The network dropped? Reconnect. Got a message we could not parse? Log it and keep going. The order book got out of sequence? Throw everything away and start over from scratch. thiserror lets us define a list of named failure types so that when something breaks, the code that catches it can look at which type it is and react correctly. Without it, we would have one generic error that tells us nothing about what to do.

axum

This is the HTTP server. When another program wants to ask what the current order book looks like, they send an HTTP request and we send back the answer. Axum handles that. We picked Axum because it is built by the same team as Tokio and works with it natively, instead of other options that have their own competing runtime underneath.

tracing and tracing-subscriber

In a normal program you would just print to see what is happening. But our program does many things simultaneously. A regular print statement has no way to tell you which of those things the message came from. tracing is logging that understands concurrency — every message knows what task it came from, what connection it was on, what it was in the middle of doing. When something goes wrong at 3am, this is how you find out why.

url

Instead of building the Binance WebSocket address by gluing strings together, we use a typed URL that validates the address as we build it. String concatenation for URLs is how you get subtle bugs — a missing slash, a wrong character — that only show up at runtime. This catches them at startup.

Tradeoff Summary

Decision	We Chose	Alternative	Why This One
Price type	rust_decimal	f64 float	Floats have rounding errors. Financial systems cannot have silent roundings.
HTTP framework	axum 0.7	actix-web	actix-web has its own actor runtime that conflicts with tokio. axum is tokio-compatible.
Error handling	thiserror	anyhow	anyhow erases error types. We need to match on error variants to know what went wrong.
Logging	tracing	println / log	println has no concept of which async task it came from. tracing tracks that.

Part 3 — error.rs: The Failure Taxonomy

We wrote the error types before any real logic. This is deliberate. If you define what can go wrong before you write the code, every piece of code knows exactly what failures it is allowed to produce. If you do it the other way around, you end up with scattered, incoherent errors that have no relationship to each other.

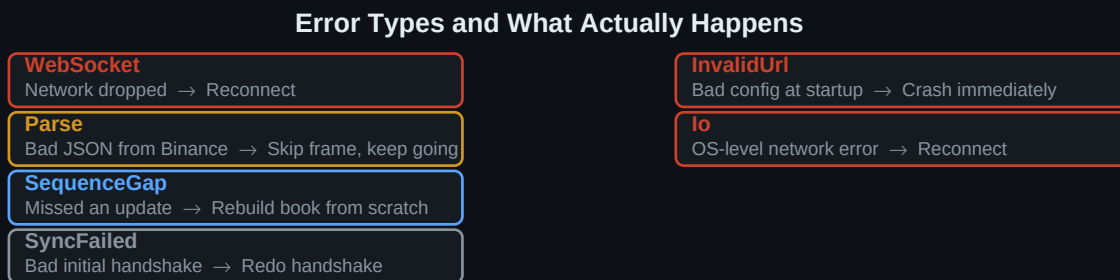


Figure 4 — Each error variant maps to a specific, different recovery action.

WebSocket

The phone call with Binance dropped. Maybe their server restarted, maybe your internet hiccupped, maybe a router between you died. When this happens: hang up and call back. The order book is still valid — we just need a new connection.

Parse

Binance sent us a message we could not understand. Maybe our code has a bug in how we described the expected shape. Maybe Binance sent something unusual. When this happens: log the bad message, throw it away, keep the connection open, keep going. One bad frame does not invalidate the entire stream.

SequenceGap

Binance does not send us the full order book every time — that would be too much data. Instead they send changes: remove this price level, add this one, update that one. Each change has a sequence number so we can tell if we missed any. If we get number 100 then number 102 without ever seeing 101, we have lost a change and our book is now wrong — we do not know what 101 was. When this happens: throw away

everything and rebuild the book from scratch.

SyncFailed

When we first connect, there is a specific handshake we do with Binance to make sure our copy of the book and their actual book are starting from the same point. If that handshake fails — if the numbers do not add up — we cannot trust anything. When this happens: start the handshake over.

InvalidUrl

The address we are trying to connect to is malformed. This only ever happens at startup if the configuration is wrong. When this happens: crash immediately with a clear message. There is no recovering from bad configuration — you need a human to fix it.

Io

Low-level network errors from the operating system itself. Connection refused, address unreachable, that kind of thing. When this happens: treat it the same as a dropped WebSocket and reconnect.

Part 4 — ws/mod.rs: The Channel Contract

The WebSocket client and the order book manager are two separate parts of the system that run independently and never call each other directly. They communicate through a queue — the client puts things in, the manager takes things out. WsEvent defines everything that can ever go in that queue. It is a contract between two parts of the system that do not otherwise know about each other.

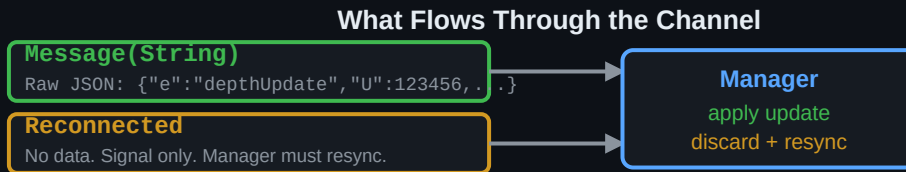


Figure 5 — Two variants, two completely different manager behaviors.

Message(String)

Binance sent us something. Here is the raw text, exactly as received. We do not parse it here — that is the manager's job. The WebSocket client's only responsibility is to get the bytes from Binance to the queue. Keeping parsing out of the client means if Binance ever changes their JSON format, we fix it in one place and the client never needs to change.

Reconnected

We lost the connection and re-established it. Whatever the manager knew before is now potentially stale — the book might be based on a stream that is no longer continuous. Start over. This signal arrives in the queue before any new messages, which is critical — if new messages arrived first, the manager would try to apply them to a book that might be from a completely different session.

Part 5 — ws/client.rs: The Reconnecting WebSocket Client

This is the piece that actually talks to Binance. It runs forever in the background, maintaining the connection and feeding messages into the queue. Here is every decision behind it.

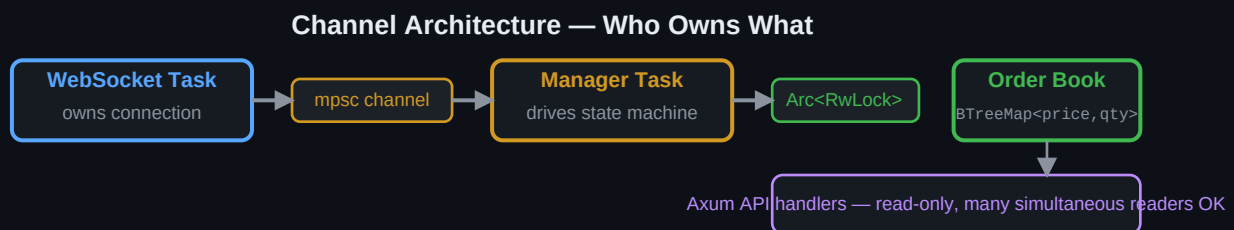


Figure 3 — One writer (manager), many readers (API). The channel enforces ordering.

Why it runs in its own independent task

When we call `spawn`, we hand the connection off to a background process and walk away. We do not sit there waiting for it to finish — it never finishes, that is the point. It runs forever in the background, feeding us messages. If we did not do it this way, the act of listening to Binance would block everything else in our program from running.

The reconnect loop — what actually happens

The outer loop is simple: try to connect. If it works, read messages until it breaks. When it breaks, try to connect again. The complexity is in how long we wait between attempts.

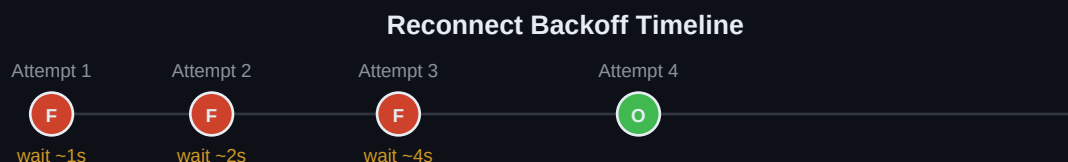


Figure 2 — Each failure doubles the wait. Jitter desynchronizes simultaneous reconnects.

Why we wait at all — and why the wait grows

Imagine Binance's servers go down. If every client immediately tries to reconnect and keeps trying every millisecond, Binance's servers come back up and immediately get flooded by millions of simultaneous reconnect attempts — which might knock them down again. We wait. And we wait longer each time we fail, because if we have failed three times in a row, something is genuinely wrong and hammering the server harder is not going to fix it.

Why we add randomness to the wait

Even with growing wait times, if ten thousand clients all started at the same moment they will all be on the same schedule. They all wait 1 second, all try at once, all fail, all wait 2 seconds, all try at once again. Adding a small random amount to each client's wait time spreads them out so they do not all hit at the exact same millisecond.

Why we cap the wait at 30 seconds

Pure doubling would eventually mean waiting hours between attempts. That is too long — if Binance comes back up after 5 minutes we want to reconnect within 30 seconds, not be stuck waiting another hour. 30 seconds is the ceiling.

The safety math — what happens after 64 failures

The wait time formula doubles each time. After 64 failed attempts, 2 to the power of 64 is a number so large it does not fit in the variable storing it. In most languages that silently wraps around to zero — no wait at all — exactly the wrong behavior when we are trying to avoid hammering a broken server. We used arithmetic that, when it would overflow, just stays at the maximum value. Then we cap it at 30 seconds. Two layers of protection.

When we send Reconnected — and why it must be first

When we successfully reconnect after a failure, the very first thing we put in the queue is Reconnected. Before any messages. This tells the manager: everything you knew is potentially wrong, start over. If we sent messages first, the manager would try to apply them to a stale book from the previous session. The ordering of events in the queue is the guarantee.

When we shut down cleanly

We check every time we try to put something in the queue whether the other side is still listening. If the manager has gone away and nobody is reading from the queue, we stop. We do not keep running a background task that produces output nobody will ever consume.

Decision	We Chose	Alternative	Why This One
Task model	spawn (fire and forget)	return JoinHandle	Returning a handle implies the caller manages the lifecycle. This task
Reconnect signal	WsEvent::Reconnected in separate control channel	Separate control channel	Two channels = possible race where a stale message arrives after th
Jitter source	System clock nanoseconds	rand crate	We only need desynchronization, not cryptographic randomness. Pul
Overflow safety	saturating_mul / saturating_checked	unchecked	After 64 failures unchecked math wraps to zero — producing instant